

Note that this assignment was done assuming $temp += x * y$ is 2 executions, one for the multiplication one for addition

1

A

Pseudo

```
Fetch_memory X[0:N-1]
Fetch_memory a
```

```
temp = 0
```

```
for(i=0 to N-1) {
    Fetch_cache a
    Fetch_cache X[i]
    temp += a * X[i]
}
```

```
write temp to y
```

$Operations = 2N$

Performance

Initial: $10 + N + 10 + 1 = N + 21$ ns

For loop: $4 * N * 0.5$ ns

Write: $10 + 1$ ns

Total Time: $3N + 32$ ns

$$P = \frac{O}{T} = \frac{2N}{3N+32} \approx \frac{2}{3} \text{ GFLOP/s}$$

Data Reuse Factor

$$Transfers = N + 2$$

$$Reuse = \frac{Operations}{Transfers} = \frac{2N}{N+2} \approx 2$$

B

Pseudo

```
Fetch_memory X[0:N-1]
Fetch_memory a

// Assume y = 0

for(i=0 to N-1) {
    Fetch_cache a
    Fetch_cache X[i]
    Fetch_memory y to register
    y += a * X[i]
    write y to Memory
}
```

$$Operations = 2N$$

Performance

$$\text{Initial: } 10 + N + 10 + 1 = N + 21 \text{ ns}$$

$$\text{Loop: } N(0.5 + 0.5 + 10 + 1 + 0.5 + 0.5 + 10 + 1) = N(24) \text{ ns}$$

$$\text{Total Time: } 25N + 21 \text{ ns}$$

$$P = \frac{O}{T} = \frac{2N}{25N+21} \approx \frac{2}{25} \text{ GFLOP/s}$$

Data Reuse Factor

$$Transfers = 3N + 1 \text{ (from } X, \text{ and } N \text{ both reads and writes for } y)$$

$$Reuse = \frac{Operations}{Transfers} = \frac{2N}{3N+1} \approx \frac{2}{3}$$

C

Pseudo

```
Fetch_memory X[0:N-1]

// Assume y = 0

for(i=0 to N-1) {
    Fetch_cache X[i]
    Fetch_memory a to register
    Fetch_memory y to register
    y += a * X[i]
    write y to Memory
}
```

$$Operations = 2N$$

Performance

$$\text{Initial: } 10 + N = N + 10 \text{ ns}$$

$$\text{Loop: } N(0.5 + 10 + 1 + 10 + 1 + 0.5 + 0.5 + 10 + 1) = N(34.5) \text{ ns}$$

$$\text{Total Time: } 35.5N + 10 \text{ ns}$$

$$P = \frac{O}{T} = \frac{2N}{35.5N+10} \approx \frac{2}{35} \text{ GFLOP/s}$$

Data Reuse Factor

$$Transfers = 4N \text{ (from } X, \text{ and } N \text{ reads for } a \text{ and } N \text{ both reads and writes for } y)$$

$$Reuse = \frac{Computations}{Operations} = \frac{2N}{4N} = \frac{1}{2}$$

2

2.1

$$A[i'][j'] = A[i][j][k], \quad i' = i, \quad j' = j * s_3 + k$$

$$B[i'][j'] = B[i][j][k], \quad i' = i * s_3 + j, \quad j' = k$$

2.2

```
for (i = 0; i < s1; i++) {
    for (j = 0; j < s4; j++) {
        C[i][j] = 0;
        for (k = 0; k < s2*s3; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

2.3

No Cache Psuedo Code + Assumptions

1. $temp += x * y$ is two executions
2. A is row major order
3. B is column major order
4. Fetch_memory goes to a register (4 registers)
5. Addition and multiplication are 1 cycle operations

```
for(i=0 to s1-1) {
    for(j=0 to s4-1) {
        temp = 0

        for(k=0 to s2*s3-1) {
            Fetch_memory A[i][k] to register
            Fetch_memory B[k][j] to register
            temp += A[i][k] * B[k][j]
        }

        write temp to C[i][j]
    }
}
```

Cache Psuedo Code + Assumptions

1. Cache is at lest $2K = 2s_2s_3$
2. $temp += x * y$ is two executions
3. A fetches replace A in cache and A in row order
4. B fetches replace B in cache abd B in column order

5. Fetch_memory goes straight to cache
6. Fetch_cache goes to a register (4 registers)
7. Addition and multiplication are 1 cycle operations

```

for(i=0 to s1-1) {
    Fetch_memory A[i][0:s2*s3-1]

    for(j=0 to s4-1) {
        Fetch_memory B[0:s2*s3-1][j]
        temp = 0

        for(k=0 to s2*s3-1) {
            Fetch_cache A[i][k]
            Fetch_cache B[k][j]
            temp += A[i][k] * B[k][j]
        }

        write temp to C[i][j]
    }
}

```

2.4

Total operations being $O = 2 * s_1 s_2 s_3 s_4$

Memory Model

Best Case

Ignore latency and every fetch is just bandwidth rate

$s_1 * s_4 * s_2 * s_3(1 + 1)$ ns inner loop and $s_1 * s_4$ ns write

$$T_B = s_1 s_4 (2s_2 s_3 + 1) \text{ ns}$$

$$P = \frac{O}{T} = \frac{2s_1 s_2 s_3 s_4}{s_1 s_4 (2s_2 s_3 + 1)} = \frac{2s_2 s_3}{2s_2 s_3 + 1} \approx 1 \text{ GFLOP/s}$$

Worst Case

Every fetch has latency

$s_1 * s_4 * s_2 * s_3(10 + 1 + 10 + 1 + 1)$ ns inner loop and $s_1 * s_4(10 + 1)$ ns write

$$T_W = s_1 s_4(23s_2 s_3 + 11) \text{ ns}$$

$$P = \frac{O}{T} = \frac{2s_1 s_2 s_3 s_4}{s_1 s_4(23s_2 s_3 + 11)} = \frac{2s_2 s_3}{23s_2 s_3 + 11} \approx \frac{2}{23} \text{ GFLOP/s}$$

Cache Model Sustained

$s_1(10 + s_2 s_3)$ ns for A

$s_1 s_4(10 + s_2 s_3)$ ns for B

$s_1 s_2 s_3 s_4(2)$ ns for inner loop

$s_1 s_4(10 + 1)$ ns for write

$$T = s_1(10 + s_2 s_3 + s_4(21 + 3s_2 s_3)) \text{ ns}$$

$$P = \frac{O}{T} = \frac{2s_1 s_2 s_3 s_4}{s_1(10 + s_2 s_3 + s_4(21 + 3s_2 s_3))} = \frac{2s_2 s_3 s_4}{10 + s_2 s_3 + s_4(21 + 3s_2 s_3)} \approx \frac{2}{3} \text{ GLOP/s}$$

2.5

A is accessed along a single row at a time so should be stored in row order

$$A(i, j) \rightarrow \text{Memory}(i * (s_2 * s_3) + j)$$

B is accessed along a single column at a time so should be stored in column order

$$B(i, j) \rightarrow \text{Memory}(i + j * (s_2 * s_3))$$

C is technically accessed along a single row at a time so should be row order

$$C(i, j) \rightarrow \text{Memory}(i * s_4 + j)$$

2.6

Assumptions:

1. A is $s_1 \times s_2 s_3$
2. B is $s_2 s_3 \times s_4$
3. Block sizes are $b_i \times b_k$ for A , $b_k \times b_j$ for B , and $b_i \times b_j$ for C
4. b_i divides s_1
5. b_k divides $s_2 s_3$
6. b_j divides s_4

```
for (i = 0 to s1/bi-1) {
  for (j = 0 to s4/bj-1) {

    // initialize bi x bj block of C to zero
    for (r = 0 to bi-1) {
      for (c = 0 to bj-1) {
        C[i*bi + r][j*bj + c] = 0;
      }
    }

    for (p = 0 to (s2*s3)/bk-1) {

      // Fetch bi rows of A, bk elements each
      for (r = 0 to bi-1) {
        Fetch A[i*bi + r][p*bk : (p+1)*bk];
      }

      // Fetch bj cols of B, bk elements each
      for (c = 0 to bj-1) {
        Fetch B[p*bk : (p+1)*bk][j*bj + c];
      }

      // Multiply the blocks and accumulate into the C block
      for (r = 0 to bi-1) {
        for (c = 0 to bj-1) {
          for (k = 0 to bk-1) {
            C[i*bi + r][j*bj + c] += A[i*bi + r][p*bk + k] * B[p*bk + k][j*bj + c];
          }
        }
      }
    }
  }
}
```

```

    // Store the bi x bj block of C back to memory
    for (r = 0 to bi-1) {
        for (c = 0 to bj-1) {
            Store C[i*bi + r][j*bj + c];
        }
    }
}

```

So for optimal block sizes, we know that $b_i b_j + b_i b_k + b_j b_k \leq S$ and $b_i \leq s+1, b_j \leq s_2 s_3, b_k \leq s_4$

We need to find data reuse factor of this algorithm so:

A is fetched $s_1 s_2 s_3 s_4 / b_j$ times, B is fetched $s_1 s_2 s_3 s_4 / b_i$ times, and C is stored $s_1 s_4$ times, so

$$Reuse = \frac{2s_1 s_2 s_3 s_4}{s_1 s_2 s_3 s_4 (1/b_i + 1/b_j) + s_1 s_4} \approx \frac{2}{1/b_i + 1/b_j}$$

Since b_i, b_j are symmetric here (due to A and B being fetched respectively) we can assume $b_i = b_j$ thus solving we get:

$b_i^2 + 2b_i b_k \leq S \rightarrow b_i = \sqrt{b_k^2 + S} - b_k$, here smaller b_k means larger b_i so larger reuse factor BUT then each fetch is smaller so there is a tradeoff.

Theres some calculus here probably to get the optimal answer BUT a good enough solution is $b_i = b_j = b_k$ so $3b^2 = S \rightarrow b = \sqrt{\frac{S}{3}}$

3

A

In row major order we can fetch with $A[i][j : j+1]$

Thus we get the following fetches:

1. $A[0][1] = 1$
2. $A[1][0 : 1] = 2, 3$
3. $A[1][2 : 3] = 4, 5$
4. $A[3][0] = 6$
5. $A[3][3] = 7$

For a total of 5 fetches

B

In column major order we can fetch with $A[i : i + 1][j]$

Thus we get the following fetches:

1. $A[0 : 1][1] = 1, 3$
2. $A[1][0] = 2$
3. $A[1][2] = 4$
4. $A[1][3] = 5$
5. $A[3][0] = 6$
6. $A[3][3] = 7$

For a total of 6 fetches

C

Since we only store the nonnzero elements, the values array would just be 7 values without any 0's separating them.

Thus we get the following fetches:

1. $C[0 : 1] = 1, 2$
2. $C[2 : 3] = 3, 4$
3. $C[4 : 5] = 5, 6$
4. $C[6] = 7$

For a total of 4 fetches

4

Original

K psuedo code

```
for (i = 0 to N/K-1) {
  for (j = 0 to N/K-1) {
    for (p = 0 to K-1) {

      // Fetch K rows of A
      for (r = 0 to K-1) {
        Fetch A[i*K + r][p*N/K : (p+1)*N/K];
      }
    }
  }
}
```

```

// Fetch K cols of B
for (c = 0 to K-1) {
    Fetch B[p*N/K : (p+1)*N/K][j*K + c];
}

// Compute K x K dot products of length N/K
for (r = 0 to K-1) {
    for (c = 0 to K-1) {
        // Assume C initialized to 0 matrix
        C[i*K + r][j*K + c] += A[i*K + r][p*N/K : (p+1)*N/K].B[p*N/K : (p+1)*N/K];
    }
}

// Store the K x K block of C back to memory
for (r = 0 to K-1) {
    for (c = 0 to K-1) {
        Store C[i*K + r][j*K + c];
    }
}
}

```

Data Reuse for this Algorithm (N and K)

$$Computations = 2 * N/K * N/K * K * K * K * N/K = 2 * N^3$$

$$Transfers = N^2/K^2 * K * N + N^2/K^2 * K * N + N^2/K^2 * K^2 = 2N^3/K + N^2$$

$$Reuse = \frac{Computations}{Transfers} = \frac{2N^3}{2N^3/K + N^2} = \frac{2N}{2N/K + 1}$$

For K=2 Same as in Class?

For $K = 2$ then we get data reuse factor as $\frac{2N}{N+1} \approx 2$ which we got on slide 37 of lecture 4.

$K=\sqrt{N}$ Have same Data Reuse as Block Matrix Multiplication?

For $K = \sqrt{N}$ then we get a data reuse factor as $\frac{2N}{2\sqrt{N}+1} = \frac{2N}{2\sqrt{N}+1} \approx \sqrt{N}$. This is really close to block matrix multiplication which had a data reuse factor of $O(\sqrt{N})$

K=1 Data Reuse Factor and What is this Algorithm?

Standard Naive matrix multiplication, row column and their dot product.

Data reuse being $\frac{2N}{2N+2} = \frac{N}{N+1} \approx 1$

Altered

Psuedo Code

Move the A fetch to outside the j loop and move the p loop in between i and j . Leave B where it is

We also can initialize C before all loops making it N^2 data transfer for init, however writing to a C block is not final so we would also need to reread C to memory

Data Reuse for this Algorithm (N and K)

$$Computations = 2 * N/K * N/K * K * K * K * N/K = 2 * N^3$$

$$Transfers = N/K * K * N + N^2/K^2 * K * N + N^2 + 2N/K * K * N/K * K * K = N^2 + N^3/K + N^2 + 2N^2K = N^3/K + 2N^2(1 + K)$$

$$Reuse = \frac{Computations}{Transfers} = \frac{2N^3}{N^3/K + 2N^2(1+K)} = \frac{2N}{N/K + 2K + 2}$$

K=2

$$Reuse = \frac{2N}{N/K + 2K + 2} = \frac{2N}{N/2 + 6} \approx 4$$

K= $\sqrt{N}$

$$Reuse = \frac{2N}{N/K + 2K + 2} = \frac{2N}{N/\sqrt{N} + 2 + 2\sqrt{N}} = \frac{2N}{3\sqrt{N} + 2} \approx \frac{2}{3}\sqrt{N}$$

5

We can model a similar algorithm where instead of each thread being incharge of a single summation between two elements and writing the sum to the latter index. Copy the same recursive doubling algorithm but instead of each thread computing a summation, we do a comparison between the numbers, and make it so the left indexed element is the smaller one and the right is the bigger one (if this isnt the case we do a nice swap using a temp variable). This effectively performs a parallel bubble sort that moves the smallest element to index 0 and the largest element to the last index

```
RMA: Read A[tid] into cache
Syncthreads()

RA: Read A[tid] into processor
Syncthreads()

for (k = 1 to log_2(N)) {
    if ((tid+1) % 2^k == 0) {
        if (A[tid] < A[tid - 2^(k-1)]) {
            Temp = A[tid - 2^(k-1)]
            A[tid - 2^(k-1)] = A[tid]
            A[tid] = Temp
        }
    }
    else if ((k != 1) and (tid % 2^k == 0)) {
        if (A[tid] > A[tid + 2^(k-1)]) {
            Temp = A[tid + 2^(k-1)]
            A[tid + 2^(k-1)] = A[tid]
            A[tid] = Temp
        }
    }
    Syncthreads()
}
```

Speedup

Serial time would be $O(N)$

This parallel algorithm is $O(\log N)$

So speedup is $O(\frac{N}{\log N})$

Scalability

This parallel algorithm doesn't get faster with more processors since only at most $N/2$ processors are active. So this is not scalable because adding more processors does not make the algorithm more efficient.

Cost

$$Cost = \#Processors \times T_p = \frac{N}{2} \times \log N \rightarrow O(N \log N)$$

Cost Optimal?

Not cost optimal because parallel algorithm work done exceeds the serial complexity of the problem: $O(N \log N) > O(N)$ so not cost optimal

6

Slightly modified from slides:

```
// In thread bid, tid

// From Sldies
RMA: Read A[bid][tid] into cache
RMB: Read B[tid] into cache
Syncthreads()
RA: Read A[bid][tid] into processor
RB: Read B[tid] into processor
M: Temp[tid] <- Mult A[bid][tid]*B[tid]
Syncthreads()

// Recursive Doubling part
for (k = 1 to log_2(N)) {
    if ((tid+1) % 2^k == 0) {
        Temp[tid] = Temp[tid] + Temp[tid - 2^(k-1)];
    }
    Syncthreads()
}

if (tid == N - 1) {
```

```
    SMC: Store Temp[N-1] -> C[bid]
}
```